



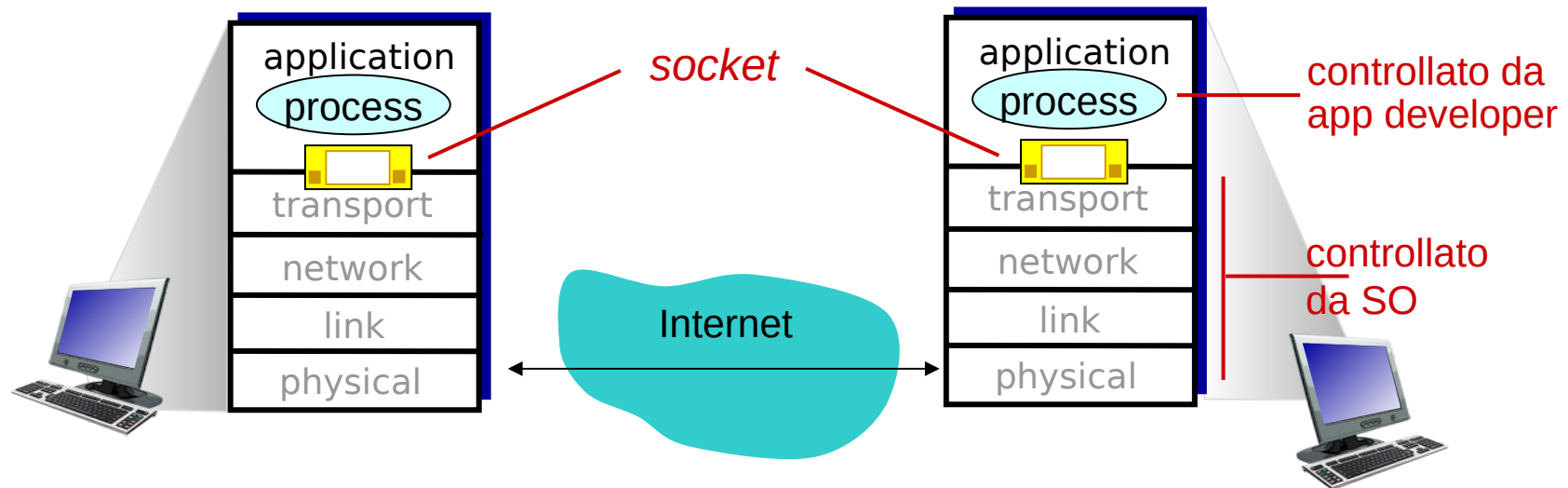
UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Socket Programming

Internet, Web e Cloud
Claudia Canali

Socket Programming

- **Obiettivo:** imparare a costruire applicazioni client/server che comunicano attraverso socket di rete
- **Socket:** interfaccia tra i processi applicativi e il livello di trasporto
- **Processi applicativi:** comunicano 'scrivendo nelle' e 'leggendo dalle' socket



E ora... 'sporchiamoci le mani!'

Socket Programming



Prima decisione da prendere

- Due tipi di socket – per due protocolli di trasporto
 - 1) UDP: non orientato a connessione, non affidabile
 - 2) TCP: affidabile, connection-oriented

Seconda decisione: protocollo applicativo?

- 3) **Applicazioni 'open'**: implementano protocolli standard (definiti in RFC) le cui regole sono note a tutti – programmi scritti in modo indipendente potranno comunicare
 - Es. client HTTP e server HTTP
- 4) **Applicazioni proprietarie**: non ci si conforma a RFC – regole definite sia per il server che per il client
 - **Numeri di porta non definiti in RFC**

Socket Programming



Semplice applicazione di rete proprietaria

- Linguaggio Python
- Prima su UDP poi su TCP
- Scrittura sia del server che del client

Applicazione di esempio

- Client: legge una linea di caratteri (dati) dalla tastiera e li invia al server
- Server: riceve i dati e converte i caratteri in maiuscole
- Server: invia i dati modificati al client
- Client: riceve i dati modificati e li mostra a video

Socket Programming – con UDP



UDP: no “connessione” tra client e server

- No handshaking prima di invio dati
- Sender: inserisce indirizzo IP destinazione e numero di porta ad ogni invio
- Receiver: estrae indirizzo IP mittente e numero di porta dal pacchetto ricevuto

Identificatore
processo
lato sender

UDP: i dati trasmessi possono essere persi o arrivare out-of-order

Punto di vista dell'applicazione

UDP offre trasporto *unreliable* di gruppi di byte (in datagrammi IP) tra client e server

Interazione Client/server su socket UDP



server

create socket UDP, port= x:

```
serverSocket =  
socket(AF_INET, SOCK_DGRAM)  
ServerSocket.bind
```

read datagram from
serverSocket

write reply to
serverSocket
specifying
client address,
port number

client

create socket UDP:

```
clientSocket =  
socket(AF_INET, SOCK_DGRAM)
```

Create message with **server IP and port=x**; send datagram via
clientSocket

read datagram from
clientSocket

close
clientSocket

Identificatore
processo
lato server

Quale porta per il server?



- Per vedere le porte già in uso sul vostro pc
- Comando **netstat**
- Netstat mostra le statistiche di rete e visualizza tutte le connessioni in entrata ed in uscita dal computer

Proto	CodaRic	CodaInv	Ind. Locale	Ind. remoto	Stato
tcp	0	0	alba:44456	server-143-204-15:https	ESTABLISHED
tcp	0	0	alba:56442	whatsapp-cdn-shv-:https	ESTABLISHED
tcp	0	0	alba:38190	lhr35s10-in-f13.1:https	ESTABLISHED

Scelta per il “nostro” server di prova - solitamente numero tra le User Ports - es. 12000

App di esempio



Python UDPClient.py

Include Python socket
library – modulo socket
alla base di tutte le
Comunicazioni di rete

Crea UDP socket lato client:
porta assegnata dal SO

Legge input da tastiera
(in formato stringa)

Attacca server name e porta,
converte il messaggio e lo
manda nella socket

Legge i byte di risposta dalla
socket

Stampa a video la
stringa e chiude la socket

```
from socket import *  
serverName = 'hostname'  
serverPort = 12000  
clientSocket = socket(AF_INET,  
                       SOCK_DGRAM)  
message = input('Inserire frase in minuscolo:')  
clientSocket.sendto(message.encode(),  
                     (serverName, serverPort))  
modifiedMessage, serverAddress =  
clientSocket.recvfrom(2048)  
print(modifiedMessage.decode())  
clientSocket.close()
```

Hostname (DNS in automatico) o IP del server

Porta su cui è in ascolto il server

Identifica una socket UDP

**Codice minimale:
Es. errori non
gestiti!!**

clientSocket = socket(AF_INET, SOCK_DGRAM)

- Primo parametro: **famiglia indirizzi**
 - AF_INET: indica che la rete sottostante usa IPv4 - Internet Protocol version 4 (livello IP)
- Secondo parametro: **tipo di socket**
 - SOCK_DGRAM → socket UDP
- Nota: non specifichiamo il numero di porta – la assegnerà il sistema operativo

clientSocket.sendto(message.encode(), (serverName, serverPort))

- message.encode() converte il messaggio **da stringa a byte** per poterlo inviare nella socket
- IP client e porta attaccati al messaggio in automatico (livelli TCP/IP)

modifiedMessage, serverAddress = clientSocket.recvfrom(2048)

- Quando un pacchetto arriva dalla rete alla socket del client, i dati (byte) sono assegnati a modifiedmessage, mentre l'indirizzo sorgente del pacchetto viene messo in serverAddress
- ServerAddress contiene sia IP che porta del server
- recvfrom(2048) prende in input la grandezza (in bit) del buffer – 2048 bit → 256 byte

modifiedMessage.decode()

- Converte i **byte** in **stringa** per la stampa

clientSocket.close()

- Chiude la socket

App di esempio



Python UDPServer.py

```
from socket import *
```

```
serverPort = 12000
```

crea UDP socket →

```
serverSocket = socket(AF_INET, SOCK_DGRAM)
```

bind ('collega') la socket
alla porta numero 12000
sul localhost →

```
serverSocket.bind('', serverPort)
```

```
print("The server is ready to receive")
```

Ciclo infinito →

```
while True:
```

Legge dalla UDP socket,
mette i dati letti in message, e
mette client IP e porta in
clientAddress →

```
message, clientAddress = serverSocket.recvfrom(2048)
```

Converte in stringa e in
maiuscolo →

```
modifiedMessage = message.decode().upper()
```

Invia indietro al client dopo
conversione da stringa a
byte →

```
serverSocket.sendto(modifiedMessage.encode(),  
clientAddress)
```

E il server si rimette in attesa...

La nostra logica
applicativa lato server!

Proviamo!



- Proviamo a lanciare sia il client che il server sul nostro PC (localhost – "127.0.0.1")
- In client_UDP.py sostituire
serverName = "hostname"
con
ServerName = "127.0.0.1" o "localhost"
- Lanciare il server
- Lanciare il client
- Scrivere una frase in minuscolo e fare invio...

Compito per casa



- **Implementare un semplice client PING basato su UDP in Python**
- Client:
 - spedirà 10 messaggi di ping a un server
 - Per ogni messaggio ricevuto in risposta dal server, deve determinare il RTT e visualizzarlo
 - Poichè UDP non è affidabile, uno dei messaggi potrebbe andare perso e il client non può aspettare all'infinito
 - Il client deve aspettare ogni risposta al massimo 1 secondo (timeout) e poi assumere che il messaggio sia andato perso e visualizzare un messaggio opportuno
- Server:
 - Ragionare sul codice
 - Basta che risponda...
 - Può simulare la perdita di qualche pacchetto (in assenza di un vero UDP – siamo su localhost)

Implementare un semplice client PING basato su UDP in Python

```
#Client
```

```
import time
```

```
clientSocket.settimeout(1.0) #Setta il timeout di un secondo
```

```
clientSocket.sendto(message,(serverName, serverPort))
```

```
try:
```

```
...
```

```
Except:
```

Scatta quando il
timeout è scaduto

#per calcolare il RTT dobbiamo calcolare il tempo passato tra l'invio del messaggio e la ricezione avvenuta (sul lato client)

```
start = time.time()
```

```
...
```

NOTA: il comando ping usa protocollo ICMP – noi implementeremo un servizio “equivalente” a ping ma molto semplice che usa socket UDP

Socket programming - con TCP



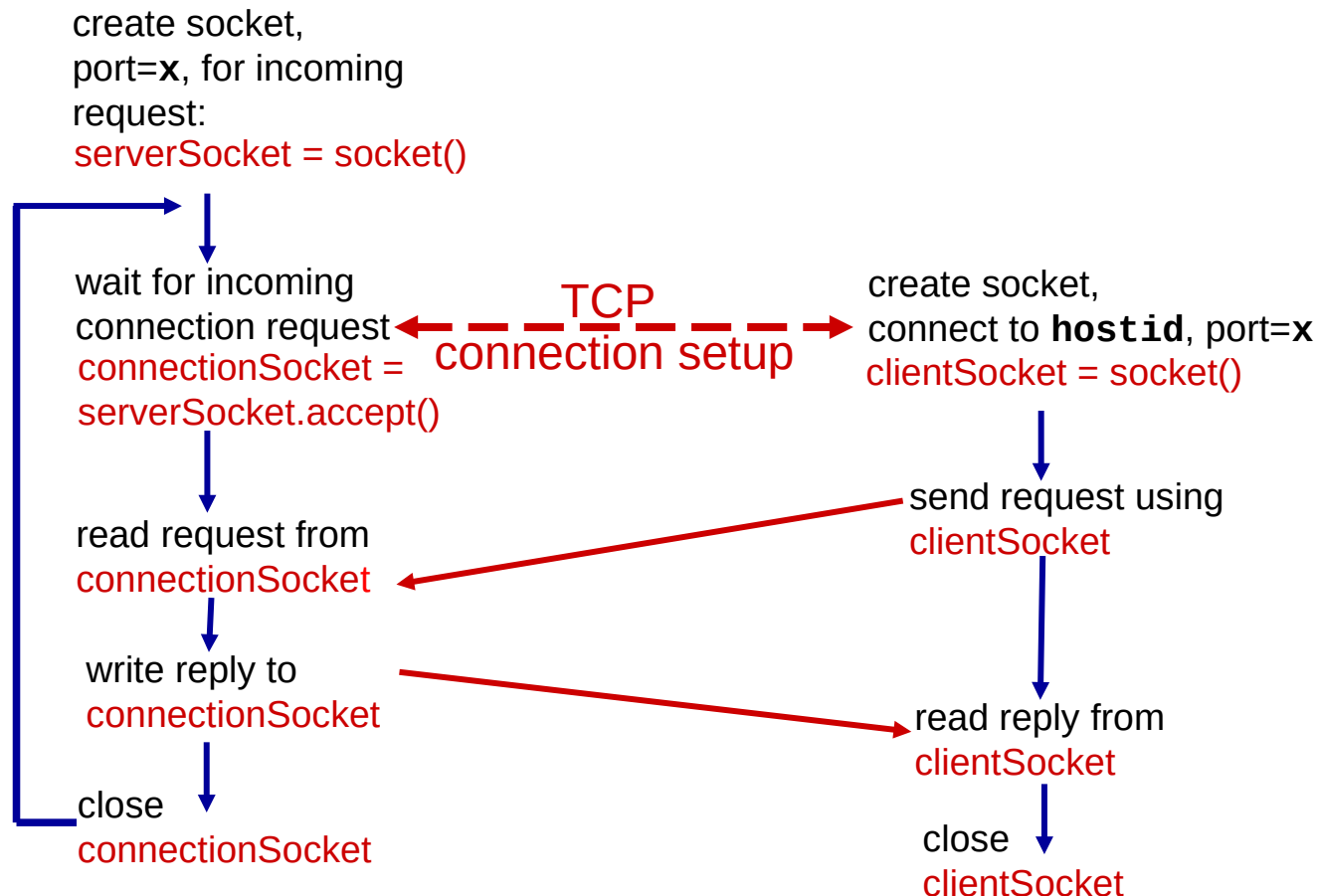
- A differenza di UDP → **necessario instaurare la connessione TCP** (three-way handhaking)
- **Il server deve creare una socket che si mette in ascolto**
- **Quando il client crea la socket TCP**, il client stabilisce la connessione TCP con il server
- Quando viene contattato dal client, il server crea una nuova socket per far comunicare il processo server con quel particolare client!
 - Permette al server di dialogare con più client in parallelo
- **Dal punto di vista dell'applicazione:**
 - TCP offre un servizio di trasporto affidabile tra client e server

Interazione client/server su socket TCP



server

client



App di esempio



Python TCPClient.py

Identifica una socket
TCP

Crea TCP socket lato client:
porta assegnata dal SO

```
from socket import *
```

```
serverName = 'hostname'
```

```
serverPort = 12000
```

```
clientSocket = socket(AF_INET, SOCK_STREAM)
```

Crea connessione TCP – prima
assente. Three-way handshaking

```
clientSocket.connect((serverName,serverPort))
```

Non c'è bisogno di attaccare
server name e porta
(connessione già instaurata)

```
sentence = input('Frase in minuscolo:')
```

```
clientSocket.send(sentence.encode())
```

In attesa della risposta
che mette in
modifiedSentence

```
modifiedSentence = clientSocket.recv(2048)
```

```
print('From Server:', modifiedSentence.decode())
```

```
clientSocket.close()
```

App di esempio



Python TCPServer.py

Identifica una socket
TCP

Crea TCP socket lato server:
serverSocket è socket di benvenuto

Bind sulla porta
12000

Si mette in ascolto di
eventuali client

Creazione di nuova socket dedicata
allo specifico client che ha richiesto
la connessione (three-way
handshaking)

```
from socket import *
SERVER_ADDRESS = "127.0.0.1"
serverPort = 12000
serverSocket = socket(AF_INET, SOCK_STREAM)
serverSocket.bind((SERVER_ADDRESS, serverPort))
serverSocket.listen(1)
print('The server is ready to receive')
while True:
    connectionSocket, addr = serverSocket.accept()
    sentence = connectionSocket.recv(2048).decode()
    capitalizedSentence = sentence.upper()
    connectionSocket.send(capitalizedSentence.encode())
    connectionSocket.close()
```